

实验四 单周期 CPU 实验

一、实验目的

- 1.理解 MIPS 指令结构, 理解 MIPS 指令集中常用指令的功能和编码, 学会对这些指令进行归纳分类。
- 2.了解熟悉 MIPS 体系的处理器结构, 如延迟槽, 哈佛结构的概念。
- 3.熟悉并掌握单周期 CPU 的原理和设计。
- 4.进一步加强运用 verilog 语言进行电路设计的能力。
- 5.为后续设计多周期 cpu 的实验打下基础。

二、实验设备

- 1.装有 Xilinx Vivado 的计算机一台
- 2.LS-CPU-EXB-002 教学系统实验箱一套

三、实验内容

单周期 CPU 就是在 1 个 CPU 周期内执行完一条指令, 执行指令所需的微操作由控制器产生, 结构简单, 容易设计。指令系统可包括如下指令类型:

运算类指令: 算术运算、逻辑运算、移位运算, 如 ADD、SUB、AND、NOT、SLL、SRL 等等。

传送类指令: 寄存器之间的传送、立即数传送, 如 MOVZ、MOVN、MFHI 等等。

访存类指令: 读存储器指令、写存储器指令, 如 LB、LW、SB、SW 等等。

控制类指令: 无条件转移、有条件转移, 如 J、JR、JAL、BEQ 等等。

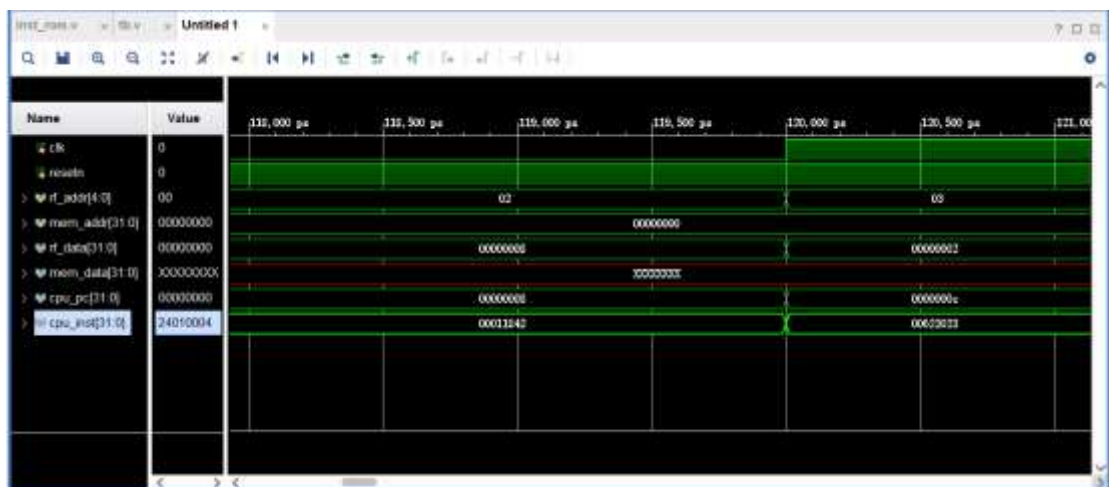
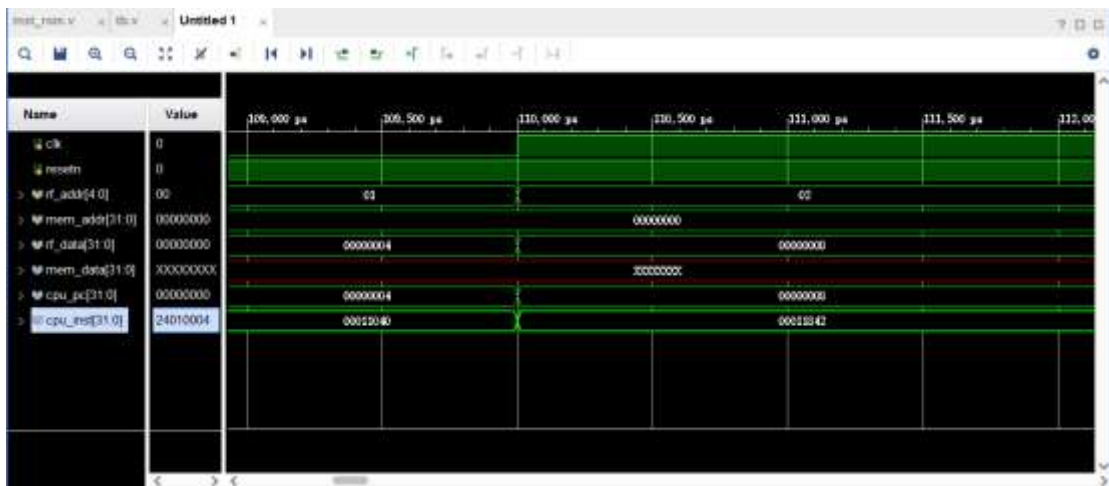
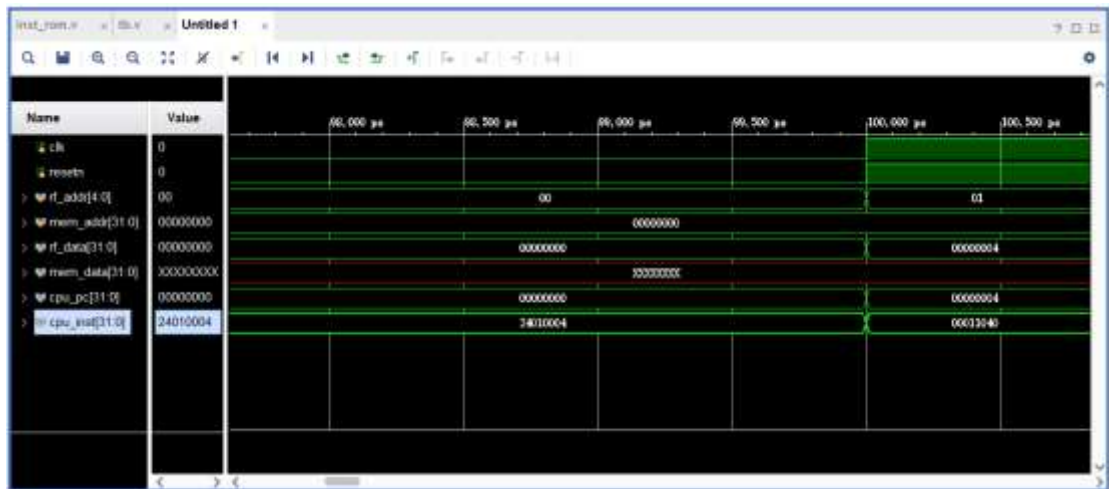
设计中所有寄存器和存储器都是异步读同步写的, 即读出数据不需要时钟控制, 但写入数据需时钟控制。故单周期 CPU 的运作即: 在一个时钟周期内, 根据 PC 值从指令 ROM 中读出相应的指令, 将指令译码后从寄存器堆中读出需要的操作数, 送往 ALU 模块, ALU 模块运算得到结果。如果是 store 指令, 则 ALU 运算结果为数据存储的地址, 就向数据 RAM 发出写请求, 在下一个时钟上升沿真正写入到数据存储。如果是 load 指令, 则 ALU 运算结果为数据存储的地址, 根据该值从数据存 RAM 中读出数据, 送往寄存器堆, 根据目的寄存器发出写请求, 在下一个时钟上升沿真正写入到寄存器堆中。如果非 load/store 操作, 若有写寄存器堆的操作, 则直接将 ALU 运算结果送往寄存器堆, 根据目的寄存器发出写请求, 在下一个时钟上升沿真正写入到寄存器堆中。如果是分支跳转指令, 则是需要将结果写入到 pc 寄存器中的。

本实验要求: 根据提供的 32 位 MIPS 框架, 至少调试出 5 条指令(运算类、传送类、访存类、控制转移类等)的运行仿真波形, 不需要上板。

四、实验步骤

- 1、启动 Vivado, 选择 File->New Project, 输入工程名称, 选择工程文件的路径
- 2、选择 RTL Project, 勾选 Do not specify sources at this time
- 3、在筛选器中进行如下选择:
family->Artix7 package->fbg676 最后选择型号 xc7a200tfbg676-2
- 4、添加源文件 Project Manager->Add sources->Add or create design sources
->Create File,开始输入程序代码
- 5、对设计文件进行仿真, 通过输入激励信号, 查看仿真后的输出信号, 判断是否符合设计要求。

五、记录实验结果。



六、撰写实验报告

tb:

```
`timescale 1ns / 1ps
```

```
////////////////////////////////////////////////////////////////
```

```
// Design Name:    single_cycle_cpu
// Module Name:
// Project Name:   single_cycle_cpu
//
// Verilog Test Fixture created by ISE for module: single_cycle_cpu
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
```

```
////////////////////////////////////////////////////////////////
```

```
module tb;
```

```
    // Inputs
    reg clk;
    reg resetn;
    reg [4:0] rf_addr;
    reg [31:0] mem_addr;
```

```
    // Outputs
    wire [31:0] rf_data;
    wire [31:0] mem_data;
    wire [31:0] cpu_pc;
    wire [31:0] cpu_inst;
```

```
    // Instantiate the Unit Under Test (UUT)
    single_cycle_cpu uut (
        .clk(clk),
        .resetn(resetn),
        .rf_addr(rf_addr),
        .mem_addr(mem_addr),
        .rf_data(rf_data),
        .mem_data(mem_data),
        .cpu_pc(cpu_pc),
        .cpu_inst(cpu_inst)
```

```

);

initial begin
    // Initialize Inputs
    clk = 1;
    resetn = 0;
    rf_addr = 0;
    mem_addr = 0;

    // Wait 100 ns for global reset to finish
    #100;
    resetn = 1;

    // Add stimulus here
    rf_addr = 1;
    #10;
    rf_addr = 2;
    #10;
    rf_addr = 3;
    #10;
    rf_addr = 4;
    #10;
    rf_addr = 5;
    #10;
    rf_addr = 6;
    mem_addr = 32'H00000000;
    #10;

end
always #5 clk=~clk;
endmodule

```

display:

```
`timescale 1ns / 1ps
```

```
//*****
```

```
//*****
```

```

module inst_rom(
    input      [4 :0] addr, // 指令地址
    output reg [31:0] inst   // 指令
);

```

```

wire [31:0] inst_rom[19:0]; // 指令存储器, 字节地址 7'b000_0000~7'b111_1111
//----- 指令编码 -----|指令地址|--- 汇编指令 -----|- 指令结果

```

-----//

```
    assign inst_rom[ 0] = 32'h24010004; // 00H: addiu $1 , $0, #4   | $1 = 0000_0004H
|
    assign inst_rom[ 1] = 32'h00011040; // 04H: sll    $2 , $1, #1   | $2 = 0000_0008H
|
    assign inst_rom[ 2] = 32'h00011842; // 0CH: srl    $3 , $1, #1   | $3 = 0000_0002H
    assign inst_rom[ 3] = 32'h00622023; // 10H: subu   $4 , $3, $2   | $4 = 0000_0006H
    assign inst_rom[ 4] = 32'h00622824; // 1CH: and    $5 , $3, $2   | $5 = FFFF_FFF3H
    assign inst_rom[ 5] = 32'h00623026; // 20H: xor    $6 , $3, $2   | $6 = 0000_0011H
    assign inst_rom[ 6] = 32'h08000000; // 4CH: j      00H           | 跳转指令 00H
```

//读指令,取 4 字节

always @(*)

begin

case (addr)

5'd0 : inst <= inst_rom[0];

5'd1 : inst <= inst_rom[1];

5'd2 : inst <= inst_rom[2];

5'd3 : inst <= inst_rom[3];

5'd4 : inst <= inst_rom[4];

5'd5 : inst <= inst_rom[5];

5'd6 : inst <= inst_rom[6];

default: inst <= 32'd0;

endcase

end

endmodule